AWS Security Hub & GuardDuty Enablement

Implementation Solution Guide

Version 1.0.0 · 2026-07-10

Contents

Executive Summary — AWS Security Hub & GuardDuty Enablement 4

What you get . . . 4

What this delivers . . . 4

The standard deployment model . . . 4

Outcome . . . 5

1. Solution Overview — AWS Security Hub & GuardDuty Enablement 5

What this solution is . . . 5

How org-wide enablement works . . . 5

The objects this solution creates (naming convention) . . . 6

What you can control . . . 6

Why this matters (the business case) . . . 6

Limitations (be honest with buyers) . . . 7

The standard deployment model . . . 7

2. Requirements — AWS Security Hub & GuardDuty Enablement 7

R1. AWS Organizations enabled with all features . . . 7

R2. A dedicated Security account to be the delegated administrator . . . 7

R3. Trusted access / service-linked permissions for both services . . . 8

R4. A chosen finding-aggregation Region . . . 8

R5. Region scope decision (which Regions to protect) . . . 8

R6. GuardDuty protection-plan and standards selection (customer decision) . . . 8

R7. Notification target and response-action policy (for the gated response) . . . 8

R8. Baseline of current security-service state . . . 9

Global Requirements — AWS (category-level) 9

G1. AWS account(s) with management/root access to the org . . . 9

G2. Least-privilege deployment identity (no long-lived root) . . . 9

G3. AWS CLI and IaC tooling . . . 9

G4. Source control + CI/CD . . . 10

G5. Terraform remote state . . . 10

G6. Region and tagging standards . . . 10

G7. Identity provider for federation (where required) . . . 10

G8. Non-production account + change control . . . 10

3. Implementation — AWS Security Hub & GuardDuty Enablement 10

Stage A — Assess . . . 11

Stage B — Configure (delegated admin + org enablement, plan-first) . . . 11

Stage C — Pilot (non-production account, one Region) . . . 12

Stage D — Validate . . . 12
Stage E — Expand (org-wide) and activate response . . . 13
5. Testing — AWS Security Hub & GuardDuty Enablement 13
Gate 1 — GuardDuty detector is enabled and analyzing . . . 13
Gate 2 — Security Hub standards are producing findings . . . 13
Gate 3 — GuardDuty findings reach Security Hub . . . 14
Gate 4 — Cross-Region / cross-account aggregation works . . . 14
Gate 5 — High-severity notification fires (response path preview) . . . 14
Gate 6 — Suppression rules remove known noise . . . 14
Gate 7 — Automated action is safe and least-privilege (only if activating) . . . 14
6. Troubleshooting — AWS Security Hub & GuardDuty Enablement 15
T1. A member account has no GuardDuty detector after enablement . . . 15
T2. A Security Hub standard won't enable or shows no controls . . . 15
T3. GuardDuty findings don't appear in Security Hub . . . 15
T4. Findings are stranded per-Region or per-account . . . 15
T5. The response path misbehaves (no notification, or the action ran when it shouldn't) . . . 15
T6. Suppression rule doesn't mute the findings . . . 16
T7. Terraform plan wants to recreate detectors/hubs you already had . . . 16
T8. Unexpected cost after enabling at scale . . . 16
7. Best Practices — AWS Security Hub & GuardDuty Enablement 16
Keep it clean . . . 16
Defer the decisions that aren't yours . . . 17
Where to go next (adjacent catalog areas) . . . 17
8. Deliverables Summary — AWS Security Hub & GuardDuty Enablement 18
Documentation . . . 18
Automation artifacts . . . 18
What the buyer receives . . . 19
Outcome . . . 19
Appendix A — PowerShell Scripts 20

Executive Summary — AWS Security Hub & GuardDuty Enablement

AWS

Most AWS environments run without their two built-in security services turned on: Amazon GuardDuty, which continuously watches account activity, network flow, and DNS for threats, and AWS Security Hub, which scores your accounts against security standards and aggregates findings into one place. Left off, threats and misconfigurations stay invisible until they become an incident, and each account is its own silo with no central view. This solution enables both services **org-wide through a delegated administrator**, turns on the CIS AWS Foundations and AWS Foundational Security Best Practices standards, aggregates findings across every account and region, and builds a triage-and-response workflow — including an automated, explicitly gated response path for high-severity findings.

This is a complete, deploy-it-carefully package for standing up AWS threat detection and posture management, written for an administrator who is new to AWS Organizations, GuardDuty, and Security Hub.

What you get

- A plain-language Solution Overview of how GuardDuty and Security Hub work, how delegated administration makes them org-wide, and how findings aggregate into a single region and queue.
- A Requirements Guide with verify / implement-if-missing for every solution-specific prerequisite (the AWS category prerequisites are published once in the separate Global Requirements).
- An Implementation Guide of precise, ordered steps — register the delegated admin, auto-enable new accounts, turn on the standards, set the aggregation Region, and wire the gated response path — always non-production first.
- Five automation artifacts: an AWS CLI discovery script, an AWS CLI validation script, an AWS CLI findings/posture report, a Terraform configuration for org-wide enablement (plan-first), and a **separately gated** Terraform configuration for the EventBridge -> Lambda/SNS automated response (disabled by default).
- Testing and Troubleshooting guides, plus a findings triage, suppression, and remediation runbook.

What this delivers

Continuous threat detection and posture scoring across the whole organization: GuardDuty analyzing every account, Security Hub scoring them against CIS and AWS FSBP, all findings collected into one aggregation Region and one prioritized queue, high-severity findings routed to a notification/response path you control, and a documented triage runbook — instead of silent, per-account blind spots.

The standard deployment model

`Assess -> Configure -> Pilot -> Validate -> Expand`, with a pilot gate that proves enablement, aggregation, and (in preview) the response path in a non-production account and one Region before auto-enabling the whole organization or activating any automated action.

*Security Hub standards content, control availability, GuardDuty finding types, and the **pricing** of both services change over time and vary by Region and partition (commercial vs GovCloud). *Verify current standards, controls, finding types, and pricing for your organization before you rely on them or enable at scale.**

Outcome

Org-wide AWS threat detection and posture management with one aggregated, actioned findings queue and a controlled automated-response path — proven on a non-production pilot before organization-wide expansion.

1. Solution Overview — AWS Security Hub & GuardDuty Enablement

What this solution is

Two native AWS security services do the heavy lifting here:

- **Amazon GuardDuty** is a continuous threat-detection service. It analyzes AWS data sources — CloudTrail management and data events, VPC flow logs, and DNS query logs (plus optional protection plans for S3, EKS, Lambda, RDS, and malware) — and raises **findings** when it spots likely-malicious or unauthorized behavior. You don't deploy agents; it reads the telemetry AWS already has.
- **AWS Security Hub** is a cloud security posture management (CSPM) and findings-aggregation service. It runs **security standards** (sets of automated controls) against your resources, gives you a **security score**, and collects findings from GuardDuty and other services into one normalized format (ASFF) and one console.

This solution turns both on across your **whole organization** using **delegated administration**, so a single security account manages detection and posture for every member account, and consolidates all findings into one **aggregation Region**.

How org-wide enablement works

1. **Delegated administrator.** From the Organizations management account you register a dedicated **Security account** as the delegated administrator for GuardDuty and for Security Hub. Day-to-day security operations then happen in that account, not the management account.
2. **Auto-enable for the organization.** In the delegated admin you turn on **auto-enable**, so GuardDuty and Security Hub (and the chosen standards) are switched on automatically for every existing and **future** member account — no per-account clicking.
3. **Standards.** In Security Hub you enable **CIS AWS Foundations Benchmark** and **AWS Foundational Security Best Practices (FSBP)**. Each standard is a bundle of controls that continuously check configuration and produce pass/fail findings that feed the security score.
4. **Finding aggregation.** GuardDuty findings flow into Security Hub. You designate one **aggregation Region** and link the other Regions, so all findings from all accounts and Regions land in a single queue.

5. **Response (gated).** An **EventBridge** rule matches high-severity Security Hub findings and routes them to **SNS** (notification) and optionally a **Lambda** (automated action). This response path is **disabled by default** and enabled only after you have validated it in preview.

The objects this solution creates (naming convention)

Every object follows `<Purpose> – <Scope> – <Stage>` so the same names appear in the steps, the scripts, the tests, and troubleshooting:

- **Security account** — the member account registered as delegated administrator for both services.
- **Aggregation Region** — the single Region where findings are consolidated.
- **EventBridge rule:** `SecFindings-HighSeverity – Org – Response` (matches high/critical Security Hub findings).
- **SNS topic:** `SecFindings-Notify – Org – Prod` (the notification target for the rule).
- **Lambda function (optional, gated):** `SecFindings-AutoRespond – Org – Gated` (automated action; deployed disabled).
- **Suppression rules (Security Hub automation rules):** named `Suppress-<control/finding> – <scope> – Prod` as you tune noise (e.g. `Suppress-CIS-1.20 – Sandbox – Prod`).

What you can control

- Which member accounts are covered (auto-enable vs selective) and which **GuardDuty protection plans** you turn on (each has its own cost).
- Which **standards** and which individual **controls** are enabled or disabled (with a documented reason).
- The single **aggregation Region** and which Regions link to it.
- Severity thresholds for the response rule, and whether the response is **notify-only** or **notify + automated action**.
- Suppression/automation rules that mute known-accepted findings so the queue stays actionable.

Why this matters (the business case)

- **Detection** — GuardDuty surfaces credential misuse, reconnaissance, crypto-mining, and exfiltration patterns that are otherwise invisible.
- **Posture** — Security Hub's CIS/FSBP score turns "are we configured safely?" into a measurable number you can drive down over time.
- **Single pane** — all findings, all accounts, all Regions, one queue — instead of logging into each account to look.
- **Controlled response** — high-severity findings notify (and optionally act) automatically, but only after you have proven the path in preview.
- **Auditability** — every object is named to a convention, so the deployment is documented and repeatable.

Limitations (be honest with buyers)

• GuardDuty and Security Hub both **bill on usage** (events/flow logs analyzed, protection plans, finding ingestion). Costs scale with account count and traffic. *Verify current pricing per Region/partition before enabling at scale.*

• Enabling the standards produces a large initial burst of findings — most are configuration gaps to triage, not active attacks. Plan triage capacity.

• This solution enables detection, posture, aggregation, and a gated response; it is **not** a full SOC, detection-engineering, or incident-response program (those build on this foundation).

• Standards content and control availability change and differ by Region/partition. *Verify current standards and controls for your organization.*

The standard deployment model

> *Assess org readiness and current security-service state, configure delegated admin + auto-enable + standards + aggregation + the gated response path, pilot in a non-production account and one Region, validate detection/aggregation/response at explicit gates, then expand org-wide and turn the response path from preview to active. Standard pattern:* `Assess -> Configure -> Pilot -> Validate -> Expand`.

2. Requirements — AWS Security Hub & GuardDuty Enablement

The AWS category prerequisites (management/root access, least-privilege deployment identity, AWS CLI + Terraform toolchain, source control + CI/CD with OIDC, remote state, region/tagging standards, an identity provider, and a non-production account with change control) are published once in **Global Requirements — AWS** and are not repeated here. Confirm those first. Below are the prerequisites **specific to enabling GuardDuty and Security Hub org-wide**. Each item: Description / Why required / How to verify / How to implement if missing.

R1. AWS Organizations enabled with all features

Description: The environment must be an AWS organization with **all features** enabled. Why required: Delegated administration and org-wide auto-enable for GuardDuty and Security Hub require Organizations with all features. How to verify: `aws organizations describe-organization` returns `"FeatureSet": "ALL"`; the console shows "All features." How to implement if missing: Enable all features from the management account, or deploy the AWS landing-zone solution first. *One-way action approved by the org owner.*

R2. A dedicated Security account to be the delegated administrator

Description: A member account (not the management account) designated to run security operations — the **Security account**. Why required: AWS best practice is to delegate GuardDuty and Security Hub administration out of the management account; the delegated admin auto-enables and manages all members. How to verify: The account exists in Organizations and you can register it as a delegated

administrator for both services. How to implement if missing: Create/designate a Security account (often in a "Security" OU) before configuring the services.

R3. Trusted access / service-linked permissions for both services

Description: GuardDuty and Security Hub must be allowed to operate across the organization (trusted access), and each account gets a service-linked role. Why required: Org-wide enablement and cross-account finding collection depend on trusted access being enabled from the management account. How to verify: In the management account the delegated-admin registration for `guardduty.amazonaws.com` and `securityhub.amazonaws.com` succeeds; service-linked roles are created on enablement. How to implement if missing: Register the delegated administrators (which enables trusted access) per the Implementation Guide; do not block the service-linked roles with SCPs.

R4. A chosen finding-aggregation Region

Description: One AWS Region selected as the **aggregation Region** where all Security Hub findings are consolidated. Why required: A single aggregation Region gives one queue for all accounts/Regions; picking it up front avoids re-linking later. How to verify: The Region is in your approved-Regions list (Global Requirements G6) and Security Hub is available there. *Verify current Region/partition availability.* How to implement if missing: Choose an aggregation Region aligned to data residency before enabling standards.

R5. Region scope decision (which Regions to protect)

Description: A decision on which Regions GuardDuty and Security Hub run in (all enabled Regions vs a defined subset). Why required: Both services are per-Region; cost and coverage depend on how many Regions you enable. Attackers use unused Regions, so unused Regions still need at least detection. How to verify: A documented list of in-scope Regions exists. How to implement if missing: The customer's security owners decide the Region scope; default to enabling detection in all Regions you can operate and centralizing findings in the aggregation Region.

R6. GuardDuty protection-plan and standards selection (customer decision)

Description: A decision on which GuardDuty protection plans (S3, EKS, Malware, Lambda, RDS, etc.) and which Security Hub standards/controls to enable. Why required: Each protection plan and standard has cost and coverage implications; enabling everything blindly creates noise and spend. How to verify: A short decision record lists the protection plans and standards (CIS AWS Foundations, AWS FSBP at minimum) and any intentionally disabled controls with reasons. How to implement if missing: The customer's security owners select the plans/standards. *This scope-and-cost determination is the customer's decision; the solution implements the selection, not the policy.* *Verify current protection plans, standards, controls, and pricing.*

R7. Notification target and response-action policy (for the gated response)

Description: An email/endpoint to subscribe to the SNS topic, and a decision on whether the response is **notify-only** or **notify + automated action**. Why required: The response path routes high-severity findings; you need a destination, and automated actions must be an explicit, approved choice. How to verify: A distribution list/endpoint exists for notifications; a decision record states whether automated remediation is permitted and for which finding types. How to implement if missing: Provide the notification endpoint. Keep

automated actions **disabled** until validated in preview and approved. *Whether automated remediation may act on production is the customer's operational/change-control decision.*

R8. Baseline of current security-service state

Description: A read of whether GuardDuty/Security Hub are already partially enabled anywhere, and any existing standards/detectors. Why required: Enabling on top of partial existing configuration can conflict; you must know the starting point. How to verify: Run the provided `AWSSEC-Discovery.sh` to report detector/hub state, standards, and delegated-admin status per account/Region. How to implement if missing: Generate the baseline before Configure; reconcile any pre-existing detectors/hubs into the delegated-admin model.

Once R1–R8 are confirmed alongside the AWS Global Requirements, proceed to the Implementation Guide. Everything is done non-production first, and the automated-response path stays in preview until validated.

Global Requirements — AWS (category-level)

These requirements apply to every solution in the AWS category (foundation and security on Amazon Web Services). Published once at the category level so listings stay small. Confirm before deploying any AWS solution. Each entry: Description / Why / Verify / Implement-if-missing.

> *AWS changes create real, billable resources across accounts. Everything runs plan/what-if first through version control, in a non-production account before production, with least-privilege credentials — never long-lived root or admin keys on a laptop.*

G1. AWS account(s) with management/root access to the org

Description: An AWS management (payer) account, or Owner-equivalent rights to create Organizations resources. Why: Landing-zone, Organizations, SCP, and delegated-admin actions operate at the org/management scope. Verify: `aws sts get-caller-identity` succeeds; the identity can access AWS Organizations. Implement: Obtain the management account (or delegated rights) before org-level changes.

G2. Least-privilege deployment identity (no long-lived root)

Description: An IAM role / IAM Identity Center permission set (or OIDC-federated CI identity) with scoped rights — not root, not static admin keys. Why: Root and long-lived admin keys are the top AWS breach vector; automation should assume a scoped role. Verify: Deployments run as an assumed role; root MFA is on and root keys do not exist. Implement: Create a scoped deployment role / permission set; enable root MFA; delete any root access keys.

G3. AWS CLI and IaC tooling

Description: AWS CLI v2, Terraform CLI (pinned), and Git installed. Why: The solutions deliver Terraform + AWS CLI; version drift breaks reproducibility. Verify: `aws --version`, `terraform version`, `git --version` return supported builds. *Verify current provider/CLI versions.* Implement: Install and pin the

toolchain; record versions in the repo.

G4. Source control + CI/CD

Description: A Git repository and a pipeline (GitHub Actions / GitLab CI / CodePipeline) with OIDC federation to AWS. Why: Plan/apply must be reviewed and run through an auditable pipeline, not locally. Verify: The pipeline can assume an AWS role via OIDC and run `terraform plan`. Implement: Configure the repo and OIDC trust; scope the CI role.

G5. Terraform remote state

Description: A remote state backend (S3 + DynamoDB lock, or Terraform Cloud). Why: Local state is unsafe and unshareable; remote state with locking prevents corruption. Verify: The state bucket/table (or TFC workspace) exists and is access-controlled and encrypted. Implement: Provision the S3 state bucket + DynamoDB lock table with encryption and versioning.

G6. Region and tagging standards

Description: Approved AWS regions, a tagging standard, and naming conventions. Why: Governance, cost allocation, and SCP/Config rules depend on consistent regions/tags. Verify: A documented standard exists and is encoded in modules/policy. Implement: Define the standard; enforce via tag policies and Config rules.

G7. Identity provider for federation (where required)

Description: Microsoft Entra ID (or another IdP) available to federate AWS IAM Identity Center. Why: SSO/SCUM federation solutions need the IdP side configured. Verify: Entra admin access to create an enterprise app (SAML + SCIM). *Verify current IAM Identity Center feature availability.* Implement: Ensure Entra rights (see the catalog's Entra solutions) before federating.

G8. Non-production account + change control

Description: A non-production account/OU to validate against, and a production change-approval process. Why: Every change is validated in non-prod and approved before production apply. Verify: A non-prod account exists; the pipeline requires approval before prod stages. Implement: Provision a non-prod account and add manual approval gates. *Production change approval is the customer's operational decision.*

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

3. Implementation — AWS Security Hub & GuardDuty Enablement

Follow these steps in order. You work in the **AWS Management Console** as an assumed least-privilege role (never root), moving between the **management account** (for delegated-admin registration) and the **Security account** (for day-to-day configuration). Every object uses the `<Purpose> – <Scope> –`

<Stage> convention from the Overview and is reused in Testing, Troubleshooting, and the scripts. **Do the pilot in a non-production account and one Region before you auto-enable the organization or activate any automated response.**

> *Portal navigation paths, standard names, control IDs, GuardDuty finding types, and pricing change and vary by Region/partition. Use the console search bar if a path has moved, and *verify current standards, controls, finding types, and pricing before enabling at scale.**

Stage A — Assess

Step A1. Baseline the current state

1. Sign in as your assumed deployment role. Run `AWSSEC-Discovery.sh` (read-only). It reports, per account and Region: whether a GuardDuty detector exists, whether Security Hub is enabled, which standards are on, and the current delegated-administrator registrations.

2. Note the **management account ID**, the **Security account ID** (R2), your chosen **aggregation Region** (R4), and the **pilot non-production account ID**.

Step A2. Confirm decisions are recorded

1. Confirm the Region scope (R5), the GuardDuty protection plans + standards selection (R6), and the notification target + response-action policy (R7) are documented. These drive the configuration variables.

Stage B — Configure (delegated admin + org enablement, plan-first)

Step B1. Register the delegated administrators (management account)

1. In the **management account**, open **Amazon GuardDuty -> Settings -> Accounts** (or **Organizations** view) and register the **Security account** as the GuardDuty **delegated administrator**.

2. Open **AWS Security Hub -> Settings -> General** and register the **Security account** as the Security Hub **delegated administrator**.

3. Registration enables trusted access for both services. From here, switch to the **Security account** for the remaining steps.

Step B2. Enable org-wide with Terraform (plan-first)

1. Use `AWSSEC-guardduty-securityhub.tf` (AWS provider). Set the input variables: aggregation Region, in-scope Regions, protection plans, standards (CIS + FSBP), and **auto-enable = true**.

2. Run **plan-first** through your pipeline/remote state (Global Requirements G4/G5): `terraform plan`. Confirm it will:

• Create/enable the GuardDuty **organization configuration** with **auto-enable for new accounts**.

• Enable **Security Hub** with **auto-enable**, subscribe the **CIS AWS Foundations** and **AWS FSBP** standards.

• Configure the **finding aggregator** pointing at the aggregation Region.

1. **Apply against the pilot non-production account and aggregation Region only** for the pilot (target the pilot resources / use a pilot workspace). Full org auto-enable is applied in Expand after the gates pass.

Step B3. Prepare the response path in preview (gated, disabled)

1. Use the **separate**, gated `AWSSEC-response.tf`. It defines:

• EventBridge rule `SecFindings-HighSeverity – Org – Response` matching high/critical Security Hub findings.

• SNS topic `SecFindings-Notify – Org – Prod` (subscribe your R7 endpoint).

• Optional Lambda `SecFindings-AutoRespond – Org – Gated`.

1. Keep the gating variable `enable_response = false` and `enable_auto_action = false` for the pilot. With these false, `terraform plan` shows the rule/topic/Lambda as **created but disabled** (rule state DISABLED, Lambda not targeted). Apply only the notification scaffolding; **do not** enable the rule or the automated action yet.

Stage C — Pilot (non-production account, one Region)

Step C1. Confirm enablement and generate a finding

1. In the **Security account**, aggregation Region, open **GuardDuty** and confirm the pilot account's detector is **enabled**.

2. Generate GuardDuty **sample findings** (**GuardDuty -> Settings -> Sample findings**, or `aws guardduty create-sample-findings`) in the pilot account.

3. Open **Security Hub -> Findings** and confirm the sample GuardDuty findings and the CIS/FSBP control findings appear.

Step C2. Confirm aggregation

1. In **Security Hub -> Settings -> Regions** (finding aggregation), confirm the aggregation Region shows linked Regions and that findings from the pilot Region are visible in the aggregation Region's queue.

Step C3. Preview the response path (no live action)

1. Temporarily enable **only the notification** by setting `enable_response = true` while keeping `enable_auto_action = false`, plan-first, and apply in the pilot.

2. Re-trigger a high-severity sample finding and confirm the SNS notification arrives at your endpoint. The Lambda automated action stays **off**.

Proceed to the Testing Guide and pass every gate before Expand.

Stage D — Validate

Run the Testing Guide (`05-testing.md`): detector enabled, standards producing findings, GuardDuty findings reaching Security Hub, cross-Region/cross-account aggregation, the high-severity notification firing, and suppression rules working. Record pass/fail. Do not expand until all gates pass.

Stage E — Expand (org-wide) and activate response

Step E1. Auto-enable the organization

1. With the gates passed, apply `AWSSEC-guardduty-securityhub.tf` at full scope (auto-enable across all member accounts and in-scope Regions), plan-first and after change approval, one Region/wave at a time if you prefer.

2. Re-run `AWSSEC-Validate.sh` after each wave to confirm every member account has a detector and Security Hub enabled with the standards on.

Step E2. Tune the queue with suppression rules

1. Triage the initial burst of findings. For known-accepted controls, create Security Hub **automation (suppression) rules** named `Suppress-<control> – <scope> – Prod` (e.g. `Suppress-CIS-1.20 – Sandbox – Prod`) so the queue reflects real risk. Record the reason for each suppression.

Step E3. Activate the response path (after approval)

1. Only after notification is proven and the response-action policy (R7) is approved, decide per finding type whether to move from **notify-only** to **notify + automated action**.

2. If automated action is approved, set `enable_auto_action = true` for the **specific, low-risk finding types** you have agreed (e.g. isolate on a confirmed pattern), plan-first, and apply. Keep the Lambda's actions least-privilege and logged.

Whether automated remediation may act on production, and for which finding types, is the customer's operational/change-control decision — activate it deliberately, per finding type, with a rollback and full logging. Never enable blanket auto-remediation on first rollout.

5. Testing — AWS Security Hub & GuardDuty Enablement

Run every gate below in the **non-production pilot account** and the **aggregation Region** before you auto-enable the organization or activate any automated response. Each gate has an explicit pass/fail. Reference the automation scripts by name where noted.

Gate 1 — GuardDuty detector is enabled and analyzing

Procedure: In the Security account (aggregation Region) open **GuardDuty**. Confirm the pilot account's detector is enabled. Optionally run `AWSSEC-Validate.sh`. Pass: The pilot account shows an **enabled** detector; `AWSSEC-Validate.sh` reports the detector enabled with the intended protection plans. Fail: No detector, or disabled. Re-check delegated-admin registration and the Terraform apply (Troubleshooting T1).

Gate 2 — Security Hub standards are producing findings

Procedure: Open **Security Hub -> Security standards**. Confirm **CIS AWS Foundations** and **AWS FSBP** are enabled and show a score/controls. Pass: Both standards are enabled and controls are evaluating (pass/fail findings and a security score appear). Fail: A standard is not enabled or shows no controls. See

Troubleshooting T2. *Verify the standard is available in your Region/partition.*

Gate 3 — GuardDuty findings reach Security Hub

Procedure: Generate GuardDuty **sample findings** in the pilot account (**GuardDuty -> Settings -> Sample findings** or `aws guardduty create-sample-findings`). Open **Security Hub -> Findings**. Pass: The GuardDuty sample findings appear in the Security Hub findings queue (normalized to ASFF). Fail: GuardDuty shows findings but Security Hub does not. Check that the GuardDuty integration is enabled in Security Hub (Troubleshooting T3).

Gate 4 — Cross-Region / cross-account aggregation works

Procedure: In **Security Hub -> Settings -> Regions**, confirm the aggregation Region with linked Regions. Confirm a finding raised in a linked Region (or a second pilot account) is visible in the aggregation Region's queue. Run `AWSSEC-Report.sh`. Pass: Findings from other Regions/accounts appear in the single aggregation queue; `AWSSEC-Report.sh` shows findings grouped across accounts/Regions. Fail: Findings are stranded per-Region/account. Re-check the finding aggregator and delegated-admin scope (Troubleshooting T4).

Gate 5 — High-severity notification fires (response path preview)

Procedure: With `enable_response = true` and `enable_auto_action = false`, re-trigger a **high/critical** sample finding. Watch your R7 SNS-subscribed endpoint. Pass: A notification for the high-severity finding arrives; the EventBridge rule `SecFindings-HighSeverity – Org – Response` shows an invocation to `SecFindings-Notify – Org – Prod`. The Lambda automated action did **not** run. Fail: No notification, or the automated action ran while it should have been off. See Troubleshooting T5.

Gate 6 — Suppression rules remove known noise

Procedure: Create a Security Hub automation rule (e.g. `Suppress-CIS-1.20 – Sandbox – Prod`) for a known-accepted control finding. Refresh the findings queue. Pass: Matching findings move to suppressed/archived and no longer clutter the active queue; the rule and its reason are recorded. Fail: Findings remain active. Check the rule's filter criteria and order (Troubleshooting T6).

Gate 7 — Automated action is safe and least-privilege (only if activating)

Procedure: Only if you are activating automated remediation (Stage E3): in the pilot, enable `enable_auto_action = true` for a **single, low-risk** finding type and trigger it. Review CloudTrail/Lambda logs. Pass: The Lambda `SecFindings-AutoRespond – Org – Gated` performs only the agreed action, logs it, and holds least-privilege permissions; nothing else is touched. Fail: The action is too broad, unlogged, or over-permissioned. Do not proceed to production until fixed (Troubleshooting T5).

When the applicable gates pass in the pilot, proceed to Expand (org-wide auto-enable, then — only after approval — activating the response path per finding type). Re-run Gate 1 and Gate 4 after each Expand wave.

6. Troubleshooting — AWS Security Hub & GuardDuty Enablement

Symptom -> cause -> fix for the failures you are most likely to hit. Work in the non-production pilot account and the aggregation Region; never troubleshoot by enabling broad automated actions in production.

T1. A member account has no GuardDuty detector after enablement

Symptom: `AWSSEC-Discovery.sh` / the console shows some accounts without an enabled detector. Cause: Delegated administration was not registered, auto-enable is off, or an SCP blocks the service-linked role. Fix: In the management account confirm the **Security account** is registered as the GuardDuty delegated administrator, and in the delegated admin confirm **auto-enable for the organization** is on (set via `AWSSEC-guardduty-securityhub.tf`). Check that no SCP denies `guardduty:*` or the service-linked role creation. Re-run the plan/apply and `AWSSEC-Validate.sh`.

T2. A Security Hub standard won't enable or shows no controls

Symptom: CIS AWS Foundations or AWS FSBP is missing, greyed out, or shows zero controls. Cause: The standard/control set is not available in that Region/partition, Security Hub isn't fully enabled there, or Config (which many controls depend on) is off. Fix: Confirm Security Hub is enabled in that Region and that **AWS Config** is recording (many controls require Config). *Verify the standard and its controls are available in your Region/partition* — availability differs between commercial and GovCloud. Re-enable the standard via Terraform.

T3. GuardDuty findings don't appear in Security Hub

Symptom: GuardDuty shows findings but the Security Hub findings queue does not include them. Cause: The GuardDuty-to-Security Hub integration is not accepted/enabled, or the accounts are managed by different delegated admins. Fix: In **Security Hub -> Integrations**, confirm **Amazon GuardDuty** is enabled (accepting findings). Ensure the **same Security account** is the delegated administrator for **both** services so findings flow into the same hub. Re-trigger a sample finding (Gate 3).

T4. Findings are stranded per-Region or per-account

Symptom: You must switch Region/account to see findings; there is no single queue. Cause: The finding aggregator isn't configured, the aggregation Region has no linked Regions, or cross-account collection isn't set in the delegated admin. Fix: In **Security Hub -> Settings -> Regions** (in the aggregation Region), configure the **finding aggregator** and link the in-scope Regions. Confirm the delegated admin manages all member accounts. Re-run `AWSSEC-Report.sh` (Gate 4).

T5. The response path misbehaves (no notification, or the action ran when it shouldn't)

Symptom: A high-severity finding produced no SNS notification, or the Lambda automated action ran while it was supposed to be off. Cause: The EventBridge rule is disabled or its pattern doesn't match; the SNS subscription is unconfirmed; or `enable_auto_action` was left true. Fix: Confirm the rule

`SecFindings-HighSeverity – Org – Response` is **enabled** (`enable_response = true`) and its event pattern matches high/critical Security Hub findings. Confirm the SNS subscription on `SecFindings-Notify – Org – Prod` is **confirmed** (recipients must click the confirmation email). For a runaway action, set `enable_auto_action = false`, plan-first, apply, and review the Lambda's CloudTrail logs. Keep the Lambda least-privilege.

T6. Suppression rule doesn't mute the findings

Symptom: Findings you created a suppression rule for are still in the active queue. Cause: The automation-rule filter criteria don't match the finding fields, or rule ordering/precedence is wrong. Fix: Re-check the automation rule's criteria against the actual finding's fields (control ID, resource, account). Confirm the rule is enabled and ordered correctly. Name it to convention (`Suppress-<control> – <scope> – Prod`) and record the reason. Re-test (Gate 6).

T7. Terraform plan wants to recreate detectors/hubs you already had

Symptom: `terraform plan` shows it will create GuardDuty detectors or Security Hub accounts that already exist (from a prior manual enablement). Cause: Pre-existing, manually enabled resources are not in Terraform state, so Terraform plans to create its own and may collide. Fix: Import the existing detector/hub into Terraform state (`terraform import`) rather than recreating, or disable the manual resources and let Terraform own them under the delegated-admin model. Reconcile the R8 baseline before org-wide apply. Always review the plan first.

T8. Unexpected cost after enabling at scale

Symptom: GuardDuty/Security Hub spend is higher than expected after org-wide enablement. Cause: High-traffic accounts, many enabled protection plans, or high finding-ingestion volume across many Regions. Fix: Review which **protection plans** are on and whether they match each account's workloads; disable plans you don't need. Confirm your Region scope (R5) is intentional. *Verify current pricing per Region/partition* and use AWS cost tooling to attribute spend by account/service.

7. Best Practices — AWS Security Hub & GuardDuty Enablement

Keep it clean

- **Delegate out of the management account.** Run detection and posture from the dedicated **Security account** delegated administrator, not the Organizations management account. The management account should hold as little operational workload as possible.
- **Auto-enable, always.** Turn on auto-enable so every existing and future member account gets GuardDuty and Security Hub automatically. New accounts are the ones most likely to be forgotten — and attacked.
- **One aggregation Region.** Consolidate all findings into a single aggregation Region so there is one queue to work. Pick it once (aligned to data residency) and link the rest.

- **Enable detection everywhere, even unused Regions.** Attackers spin up resources in Regions you never use. At minimum keep GuardDuty on in all Regions you can operate; centralize the findings.
- **Terraform is the source of truth.** Make enablement, standards, and the response path through `AWSSEC-guardduty-securityhub.tf` and `AWSSEC-response.tf`, plan-first and reviewed. Console clicks drift and are hard to audit.
- **Treat the response path as loaded until proven.** Keep `enable_response`/`enable_auto_action` false until you have validated notification in preview. Automated remediation is powerful and dangerous — enable it deliberately, per finding type, with logging and rollback.
- **Triage the initial burst; suppress with reasons.** Enabling standards produces many configuration findings at once. Work them down and use named suppression rules (`Suppress-<control> – <scope> – Prod`) with a recorded justification for anything you mute — never suppress blindly.
- **Drive the score down over time.** The Security Hub score is a trend, not a one-time number. Track it per account and remediate the highest-severity failing controls first.
- **Right-size protection plans.** Each GuardDuty protection plan (S3, EKS, Malware, Lambda, RDS) adds cost and coverage. Enable the ones that match your workloads; don't pay for what you don't run.
- **Watch the bill.** Both services bill on usage and it scales with account count and traffic. Review cost after enabling at scale. *Verify current pricing per Region/partition.*
- **Re-verify facts.** Standards content, control IDs, GuardDuty finding types, Region availability, and pricing all change. Before relying on a specific control or finding type, *verify current availability for your organization.*

Defer the decisions that aren't yours

Which standards and protection plans to run, which controls may be disabled or suppressed, whether automated remediation may act on production and for which finding types, and how findings map to your incident-response process are security-governance and change-control decisions for the customer's security and operations owners. This solution provides the detection, posture, aggregation, and a gated response mechanism — not the risk-acceptance or remediation-authority policy.

Where to go next (adjacent catalog areas)

- **AWS landing zone / Organizations baseline** — the OU structure, SCPs, and account factory that this security layer monitors across.
- **AWS IAM Identity Center + Entra SSO federation** — federate least-privilege access so the accounts you are now monitoring are also accessed safely.
- **Microsoft Sentinel telemetry pipelines** — forward AWS Security Hub findings into a central SIEM if your SOC standardizes on Sentinel.
- **Incident-response runbook** — build the response playbooks that the high-severity findings and the gated automation feed into.

8. Deliverables Summary — AWS Security Hub & GuardDuty Enablement

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	How GuardDuty and Security Hub work, delegated administration, standards, finding aggregation, the gated response path, the named objects, scope, and the deployment model
2. Requirements Guide	Solution-specific prerequisites with Verify / Implement-if-missing
Global Requirements (AWS)	Category-wide AWS prerequisites, published once
3. Implementation Guide	Exact, ordered steps — delegated admin, org auto-enable, standards, aggregation, gated response, pilot, expand, activate
5. Testing Guide	Seven pass/fail validation gates (detector, standards, GuardDuty->Hub, aggregation, high-severity notification, suppression, safe automation)
6. Troubleshooting Guide	Symptom -> cause -> fix for the common enablement and response failures
7. Best Practices	Keep-it-clean guidance and where to go next

Automation artifacts

Script	What it does
`AWSSEC-Discovery.sh`	AWS CLI, read-only. Reports GuardDuty detector state, Security Hub enablement, enabled standards, and delegated-admin registrations per account/Region (the R8 baseline)
`AWSSEC-Validate.sh`	AWS CLI, read-only. Confirms each in-scope account has a GuardDuty detector and Security Hub enabled with CIS/FSBP on and the finding aggregator configured
`AWSSEC-Report.sh`	AWS CLI, read-only. Produces a findings/posture report — counts by severity, standard scores, and findings grouped across accounts/Regions

Script	What it does
`AWSSEC-guardduty-securityhub.tf`	Terraform (AWS provider). Org-wide enablement: delegated-admin org config, auto-enable, CIS + FSBP standards, and the finding aggregator. Run **plan-first**; pilot scope first, org-wide after approval
`AWSSEC-response.tf`	Terraform (AWS provider). The **gated** response path — EventBridge rule -> SNS -> optional Lambda. Ships **disabled** (`enable_response`/`enable_auto_action` false); notification then automated action are enabled deliberately after validation

All apply/change artifacts are plan/what-if-first and pilot-first: Terraform runs `plan` before `apply`, the response path is disabled by default, and every discovery/validation/report script is read-only.

What the buyer receives

- This implementation guide as a Word document and PDF.
- All five automation artifacts (AWS CLI `.sh`, Terraform `.tf`) in a single ZIP — delivered as a Complete Solution Package (PDF + Word + scripts).
- A repeatable process following the standard model: `Assess -> Configure -> Pilot -> Validate -> Expand`.

Outcome

Org-wide AWS threat detection and posture management — GuardDuty and Security Hub enabled through a delegated admin, CIS/FSBP standards scoring every account, findings aggregated into one queue, and a controlled automated-response path — proven on a non-production pilot before organization-wide expansion.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

AWSSEC-Discovery.sh

Purpose: Reports the starting point before you enable org-wide: whether a GuardDuty detector exists per region, whether Security Hub is enabled, which standards are on, and the current delegated-administrator registrations. Read-only — it creates and changes nothing. Reconcile any pre-existing detectors/hubs into the delegated-admin model before org-wide apply. HOW TO RUN Authenticate first as your least-privilege role (never root): aws sts get-caller-identity Then (optionally pass a space-separated region list; defaults to the CLI region): ./AWSSEC-Discovery.sh "us-east-1 us-west-2 eu-west-1" Requires: AWS CLI v2, jq. Rights: read on guardduty, securityhub, organizations. EXPECTED RESULTS A printed and saved report per region: detector id + status, Security Hub enablement, enabled standards, and delegated-admin accounts. NOTES White-label. Read-only. Safe to run repeatedly (e.g. after each Expand wave). Verify current standards/finding-types availability per region/partition.
========== set -euo pipefail

How to run: Authenticate first as your least-privilege role (never root): aws sts get-caller-identity Then (optionally pass a space-separated region list; defaults to the CLI region): ./AWSSEC-Discovery.sh "us-east-1 us-west-2 eu-west-1" Requires: AWS CLI v2, jq. Rights: read on guardduty, securityhub, organizations. EXPECTED RESULTS A printed and saved report per region: detector id + status, Security Hub enablement, enabled standards, and delegated-admin accounts. NOTES White-label. Read-only. Safe to run repeatedly (e.g. after each Expand wave). Verify current standards/finding-types availability per region/partition.
========== set -euo pipefail

AWSSEC-Report.sh

Purpose: Produces a triage-ready snapshot from the Security Hub aggregation Region: active-finding counts by severity, enabled-standard scores, and the top findings grouped across accounts. Use it to drive the triage runbook and for Testing Gate 4 (aggregation) and Gate 6 (suppression) evidence. Read-only — creates and changes nothing. HOW TO RUN Authenticate as your least-privilege role, then run in the AGGREGATION region: ./AWSSEC-Report.sh us-east-1 Requires: AWS CLI v2, jq. Rights: read on securityhub. EXPECTED RESULTS Printed and saved: - Active findings by severity (CRITICAL/HIGH/MEDIUM/LOW) - Per-standard control scores - Top active findings (title, severity, account, resource) NOTES White-label. Read-only. Only ACTIVE, non-suppressed findings are counted.
========== set -euo pipefail

How to run: Authenticate as your least-privilege role, then run in the AGGREGATION region: ./AWSSEC-Report.sh us-east-1 Requires: AWS CLI v2, jq. Rights: read on securityhub. EXPECTED RESULTS Printed and saved: - Active findings by severity (CRITICAL/HIGH/MEDIUM/LOW) - Per-standard control scores - Top active findings (title, severity, account, resource) NOTES White-label. Read-only. Only ACTIVE, non-suppressed findings are counted.
========== set -euo pipefail

AWSSEC-Validate.sh

Purpose: Confirms the enablement is healthy in a given region: a GuardDuty detector is enabled with org auto-enable, Security Hub is enabled, the CIS AWS Foundations and AWS FSBP standards are on, and a finding aggregator exists. Prints PASS/WARN per check. Read-only — changes nothing. Run in the pilot region during Validate, and after each Expand wave. HOW TO RUN Authenticate as your least-privilege role, then: ./AWSSEC-Validate.sh us-east-1 Requires: AWS CLI v2, jq. Rights: read on guardduty, securityhub. EXPECTED RESULTS A checklist: [PASS] GuardDuty detector enabled [PASS] Org auto-enable on [PASS] Security Hub enabled [PASS] CIS AWS Foundations standard enabled [PASS] AWS FSBP standard enabled [PASS] Finding aggregator configured WARN items point to the matching Troubleshooting entry. NOTES White-label. Read-only. Verify current standard ARNs/availability per region/partition.
========== set -euo pipefail

How to run: Authenticate as your least-privilege role, then: ./AWSSEC-Validate.sh us-east-1 Requires: AWS CLI v2, jq. Rights: read on guardduty, securityhub. EXPECTED RESULTS A checklist: [PASS] GuardDuty detector enabled [PASS] Org auto-enable on [PASS] Security Hub enabled [PASS] CIS AWS Foundations standard enabled [PASS] AWS FSBP standard enabled [PASS] Finding aggregator configured WARN items point to the matching Troubleshooting entry. NOTES White-label. Read-only. Verify current standard ARNs/availability per region/partition.
========== set -euo pipefail

AWSSEC-guardduty-securityhub.tf

Purpose: Declares, as code: the GuardDuty organization configuration with auto-enable, Security Hub with auto-enable, the CIS AWS Foundations and AWS FSBP standards, and the finding aggregator pointing at your aggregation Region. Run from the delegated-admin (Security) account. HOW TO RUN (PLAN-FIRST, always) 1. Configure the AWS provider to ASSUME your least-privilege role in the Security (delegated-admin) account, plus a remote backend (S3 + lock). 2. terraform init 3. terraform plan -var-file=pilot.tfvars # REVIEW; pilot scope first 4. terraform apply -var-file=pilot.tfvars # after approval (Org-wide auto-enable + more Regions in a later var-file, per Expand wave.) EXPECTED RESULTS GuardDuty detector + org auto-enable, Security Hub enabled with CIS + FSBP, and a finding aggregator linking the in-scope Regions. NOTES White-label. Plan-first / pilot-first. Standard ARNs, control availability, protection plans, and pricing change and vary by Region/partition — VERIFY current values before enabling at scale. The automated-response path is in a SEPARATE, gated file: AWSSEC-response.tf.
==========

AWSSEC-response.tf

Purpose: Declares the response scaffolding as code, but SHIPS DISABLED. Two gates control it: enable_response (default false) — turns the EventBridge rule ON so high/critical findings NOTIFY via SNS. enable_auto_action (default false) — additionally routes to a Lambda that performs an approved automated action. Pilot: keep both false (rule created but DISABLED). Validate notification by setting only enable_response=true. Activate automated action (per finding type, after approval) with enable_auto_action=true — NEVER on first rollout. HOW TO RUN (PLAN-FIRST, always) 1. Configure the AWS provider to ASSUME your least-privilege role in the Security account; use remote state. 2. terraform init 3. terraform plan -var-file=pilot.tfvars # both gates false = disabled 4. terraform apply

-var-file=pilot.tfvars Later: -var 'enable_response=true' (notify-only, validate) -var 'enable_auto_action=true' (per approved finding type ONLY) EXPECTED RESULTS SNS topic + subscription, an EventBridge rule (DISABLED unless enable_response), and an optional Lambda (created only if enable_auto_action). NOTES White-label. GATED and disabled by default. Whether automated remediation may act on production is the customer's operational/change decision. Keep the Lambda least-privilege and fully logged. Verify current finding schema/fields.
==========